

HealthSync — SaaS Roadmap & First Client Playbook

From working portfolio app to first paying Egyptian clinic client.

Table of Contents

- [1. Current State](#)
 - [2. Phase 2 — Replace json-server with Supabase](#)
 - [3. Phase 3 — Real Auth + Multi-tenancy](#)
 - [4. Phase 4 — Billing + Onboarding](#)
 - [5. Phase 5 — First Client Acquisition](#)
 - [6. Feature Priority Map — Egyptian Market](#)
 - [7. Full Tech Stack Reference](#)
 - [8. Pricing Strategy](#)
-

1. Current State

Status: Phase 1 complete.

HealthSync is a fully built, deployed Angular 17 SPA. You are not starting from zero — you are starting from a working product.

What is already built

Module	Status	Notes
Admin dashboard	Done	Metrics, today's schedule, quick actions
Appointments management	Done	Full CRUD, status management
Patient management	Done	Profiles, search, history
Doctor management	Done	Schedules, departments
Medical records	Done	Timeline, diagnoses, vitals
Prescriptions	Done	Medications, dosage, duration
Analytics dashboard	Done	Chart.js, lazy-loaded
Multi-role auth	Done (mock)	Admin, Doctor, Patient, Receptionist
Deployment	Done	Vercel (frontend) + Railway (json-server backend)

What is missing for a real SaaS

- Real database (Supabase replaces json-server)
 - Real authentication (Supabase Auth replaces mock)
 - Multi-tenancy (each clinic sees only their own data)
 - Clinic registration and onboarding flow
 - Billing and subscription management (Stripe)
 - Egypt-specific features: WhatsApp reminders, Arabic UI, cash tracking, self-booking
-

2. Phase 2 — Replace json-server with Supabase

Estimated time: 2 weeks (evenings)

json-server is a development tool. It resets on redeploy, has no authentication, and cannot isolate data between multiple clinics. Supabase provides a real PostgreSQL database, built-in authentication, and row-level security — all on a free tier up to 500MB.

Step 1 — Create a Supabase project

1. Go to supabase.com and create a free account.
2. Create a new project. Choose the Frankfurt region (closest to Egypt with good latency).
3. Copy your project URL and anon public key from **Settings** → **API**.
4. Add them to your Angular environment files:

```
// src/environments/environment.ts
export const environment = {
  production: false,
  supabaseUrl: 'https://your-project.supabase.co',
  supabaseAnonKey: 'your-anon-key'
};
```

Step 2 — Install Supabase client

```
npm install @supabase/supabase-js
```

Create a singleton service:

```
// src/app/services/supabase.service.ts
import { Injectable } from '@angular/core';
import { createClient, SupabaseClient } from '@supabase/supabase-js';
import { environment } from '../../environments/environment';

@Injectable({ providedIn: 'root' })
export class SupabaseService {
  client: SupabaseClient = createClient(
    environment.supabaseUrl,
    environment.supabaseAnonKey
  );
}
```

Step 3 — Database schema

Run the following SQL in **Supabase → SQL Editor**. This creates all tables with multi-tenancy built in via `clinic_id`.

```
-- Clinics (one row per tenant)
create table clinics (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text unique not null,          -- used for public booking URL
  phone text,
  address text,
  plan text default 'trial',          -- trial | starter | pro
  trial_ends_at timestamptz default now() + interval '14 days',
  created_at timestamptz default now()
);

-- Users (doctors, admins, receptionists, patients)
create table users (
  id uuid primary key references auth.users on delete cascade,
  clinic_id uuid references clinics(id) on delete cascade,
  full_name text not null,
  role text not null check (role in
('admin','doctor','receptionist','patient')),
  phone text,
  specialization text,                -- doctors only
  created_at timestamptz default now()
);

-- Patients
create table patients (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid references clinics(id) on delete cascade,
  full_name text not null,
  phone text,
  date_of_birth date,
  gender text,
  blood_type text,
  no_show_count int default 0,
  created_at timestamptz default now()
);

-- Appointments
create table appointments (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid references clinics(id) on delete cascade,
  patient_id uuid references patients(id),
  doctor_id uuid references users(id),
  scheduled_at timestamptz not null,
  type text,                          -- checkup | follow-up | consultation
  status text default 'pending' check (status in
('pending','confirmed','completed','cancelled','no_show')),
  fee numeric(10,2),
  paid boolean default false,
  paid_at timestamptz,
  notes text,
  created_at timestamptz default now()
);

-- Medical records
create table medical_records (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid references clinics(id) on delete cascade,
```

```

    patient_id uuid references patients(id),
    doctor_id uuid references users(id),
    appointment_id uuid references appointments(id),
    diagnosis text,
    notes text,
    follow_up_date date,
    created_at timestamptz default now()
);

-- Prescriptions
create table prescriptions (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid references clinics(id) on delete cascade,
  record_id uuid references medical_records(id),
  medication text not null,
  dosage text,
  frequency text,
  duration_days int,
  notes text
);

-- Payments
create table payments (
  id uuid primary key default gen_random_uuid(),
  clinic_id uuid references clinics(id) on delete cascade,
  appointment_id uuid references appointments(id),
  amount numeric(10,2) not null,
  method text default 'cash',
  received_at timestamptz default now(),
  notes text
);

```

Step 4 — Enable Row Level Security (RLS)

RLS ensures each clinic can only see its own data. Run for every table:

```

-- Example for appointments (repeat pattern for all tables)
alter table appointments enable row level security;

create policy "clinic members only"
on appointments for all
using (
  clinic_id = (
    select clinic_id from users where id = auth.uid()
  )
);

```

Repeat for: patients, medical_records, prescriptions, payments, users.

Step 5 — Swap Angular services

Replace your existing HTTP calls with Supabase client calls. Example for appointments:

```

// Before (json-server)
getAppointments(): Observable<Appointment[]> {
  return this.http.get<Appointment[]>(`${this.apiUrl}/appointments`);
}

// After (Supabase)
async getAppointments(): Promise<Appointment[]> {
  const { data, error } = await this.supabase.client
    .from('appointments')

```

```

        .select('*', patients(*), users!doctor_id(*)')
        .order('scheduled_at', { ascending: true });
    if (error) throw error;
    return data;
}

```

Wrap async calls in Angular with `from()` from RxJS to keep the Observable pattern if preferred.

3. Phase 3 — Real Auth + Multi-tenancy

Estimated time: 1 week

Step 1 — Enable Supabase Auth

In Supabase dashboard: **Authentication** → **Providers** → **Email** (enable it). This gives you email/password sign-in out of the box.

Step 2 — Clinic registration flow

New clinic owners sign up and create their clinic in one flow:

```

// registration.service.ts
async registerClinic(email: string, password: string, clinicName: string,
ownerName: string) {
    // 1. Create auth user
    const { data: authData, error: authError } = await
this.supabase.client.auth.signUp({
    email, password
});
    if (authError) throw authError;

    // 2. Create clinic row
    const slug = clinicName.toLowerCase().replace(/\s+/g, '-');
    const { data: clinic, error: clinicError } = await this.supabase.client
        .from('clinics')
        .insert({ name: clinicName, slug })
        .select()
        .single();
    if (clinicError) throw clinicError;

    // 3. Create user profile as admin
    await this.supabase.client.from('users').insert({
        id: authData.user!.id,
        clinic_id: clinic.id,
        full_name: ownerName,
        role: 'admin'
    });

    return clinic;
}

```

Step 3 — Update Angular auth guards

Replace mock auth guards with real JWT-based guards:

```

// auth.guard.ts
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {

```

```

constructor(private supabase: SupabaseService, private router: Router) {}

async canActivate(): Promise<boolean> {
  const { data: { session } } = await this.supabase.client.auth.getSession();
  if (!session) {
    this.router.navigate(['/login']);
    return false;
  }
  return true;
}
}

```

Step 4 — Staff invite flow

Admin invites a doctor or receptionist by email:

```

// Uses Supabase's built-in invite functionality
async inviteStaff(email: string, role: string, clinicId: string) {
  const { data, error } = await
this.supabase.client.auth.admin.inviteUserByEmail(email, {
    data: { clinic_id: clinicId, role }
  });
  // Invited user receives an email with a magic link
  // On first login, create their users row using the metadata
}

```

Step 5 — Validate data isolation

Create two test clinics, log into each, and confirm you cannot see the other's patients or appointments. This must pass before moving to Phase 4.

4. Phase 4 — Billing + Onboarding

Estimated time: 1 week

Pricing decision

Start with one simple plan. Add tiers only after you have paying clients.

Plan	Price	Limits
Starter	200 EGP/month (~\$4)	Up to 3 doctors
Pro	450 EGP/month (~\$9)	Unlimited doctors + WhatsApp reminders

For UAE or remote clients, price in USD (\$15/\$35 per month).

Step 1 — Create Stripe account

1. Go to stripe.com. Create an account.
2. Create two products in the Stripe dashboard: Starter and Pro.
3. Copy the price IDs (they look like `price_1abc...`).

Step 2 — Stripe Checkout integration

```
// billing.service.ts
async createCheckoutSession(priceId: string, clinicId: string) {
  const response = await fetch('/api/create-checkout-session', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ priceId, clinicId })
  });
  const { url } = await response.json();
  window.location.href = url; // Redirect to Stripe hosted checkout
}
```

Use a Supabase Edge Function (or a small Express server) as the backend for the checkout session:

```
// supabase/functions/create-checkout-session/index.ts
import Stripe from 'stripe';
const stripe = new Stripe(Deno.env.get('STRIPE_SECRET_KEY')!);

Deno.serve(async (req) => {
  const { priceId, clinicId } = await req.json();
  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    payment_method_types: ['card'],
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `https://healthsync.vercel.app/billing/success`,
    cancel_url: `https://healthsync.vercel.app/billing`,
    metadata: { clinicId }
  });
  return new Response(JSON.stringify({ url: session.url }));
});
```

Step 3 — Stripe webhook

When a clinic pays, Stripe calls your webhook. Update the clinic's plan in Supabase:

```
// Handle subscription created / cancelled
if (event.type === 'customer.subscription.created') {
  const clinicId = event.data.object.metadata.clinicId;
  await supabase.from('clinics').update({ plan: 'starter' }).eq('id', clinicId);
}
if (event.type === 'customer.subscription.deleted') {
  await supabase.from('clinics').update({ plan: 'expired' }).eq('id', clinicId);
}
```

Step 4 — Trial gate in Angular

Add a trial expiry banner and block write operations when trial or plan is expired:

```
// In your AppComponent or a TenantService
async checkPlanStatus() {
  const clinic = await this.clinicService.getCurrentClinic();
  const trialExpired = new Date(clinic.trial_ends_at) < new Date();
  const planExpired = clinic.plan === 'expired';

  if (trialExpired && planExpired) {
    this.showUpgradeBanner = true;
    this.disableWrites = true;
  }
}
```

Step 5 — Landing page

Build a simple one-page site (can be a separate Angular route or a plain HTML page):

- Headline in Arabic + English: "إدارة عيادتك بسهولة" / "Run your clinic simply"
 - 3 key benefits: no-shows reduced, easy scheduling, instant receipts
 - Pricing section (Starter / Pro)
 - WhatsApp button linking to your number: <https://wa.me/20XXXXXXXXXX>
 - Screenshots of the actual product
-

5. Phase 5 — First Client Acquisition

Start this NOW, in parallel with Phase 2. Do not wait for the product to be "ready".

The Vercel demo is good enough to show. If you can get a clinic owner to say "yes I'd use this", that validates the product before you spend 5 more weeks building.

Week 1 — Research

1. Identify 20 small private clinics in Giza and central Cairo. Focus on:
 - General practice (GP) clinics
 - 1–5 doctors
 - No existing software (still on paper/Excel)
2. Find their Facebook pages or phone numbers.
3. Note whether they post their schedule publicly (an opening for "what if patients could book online?").

Week 2 — Outreach

In-person (highest conversion): Walk in during a slow period (late morning or early afternoon, not peak hours). Talk to the receptionist first, not the doctor. Script:

أقدر أريكم demo السلام عليكم، أنا بشتغل على برنامج لإدارة المواعيد للعيادات. عندي "وتتبع المدفوعات no-shows إياه في 10 دقائق؟ يساعد على تقليل الـ"

Translation: "I'm building an appointment management system for clinics. Can I show you a 10-minute demo? It helps reduce no-shows and track payments."

WhatsApp outreach:

اسمي كريم ويعمل على برنامج لإدارة العيادات. مش هضيق، [Name] السلام عليكم دكتور من وقتك — بس كنت عايز أعرف هل عندك مشكلة في تنظيم المواعيد أو متابعة المدفوعات؟ لو إيه، عندي حاجة ممكن تفيدك.

Facebook groups to post in:

- أطباء مصر
- أطباء الجيزة
- ممارسو الصحة في مصر

Post: offer 3 months free to beta clinics in exchange for feedback.

Week 3 — Demo and close

When you get a meeting:

1. **Open with their pain, not your product:** "How do you currently manage appointments? What's the biggest headache?"
2. **Show the relevant feature first.** If they complain about no-shows → show WhatsApp reminders. If they complain about tracking money → show the payment tracking screen.
3. **Do not list features.** Show one flow end-to-end: patient books → receptionist confirms → automatic WhatsApp reminder sent.
4. **Handle objections** (see table below).
5. **Close with a free trial offer:** "I'll set everything up for you. You don't pay anything for 30 days. If it doesn't save you time, you stop — no commitment."

Common objections and responses

Objection	Response
"Our patients aren't tech-savvy"	"The patient doesn't need to do anything. The receptionist uses it. Patients just receive a WhatsApp — they already know how to read WhatsApp."
"We already have a system"	"What system? How long does it take to book one appointment in it?" (Most will say Excel or paper.)
"It looks too complicated"	"I'll set it up and train your receptionist in one hour. I'll be available on WhatsApp for any questions."
"Is it secure? Patient data is sensitive"	"All data is stored on encrypted servers in Europe (Supabase uses AWS Frankfurt). I can show you the documentation."
"We'll think about it"	"Can I set up the demo account with your actual doctors and schedule? That way you can try it with real data, with no commitment."

After the first yes

1. **Onboard them manually.** Set up their clinic yourself. Enter their doctors, set their schedule, import any existing patient data from their Excel/notebook. This is normal for the first 5 clients.

2. **Visit once a week** for the first month. Fix problems immediately.
 3. **Ask for a testimonial** after 30 days.
 4. **Ask for a referral.** "Do you know any other doctors who might find this useful?" Word of mouth in medical communities in Egypt is very strong.
-

6. Feature Priority Map — Egyptian Market

Tier 1 — Must-have (build these before selling)

WhatsApp appointment reminders

Why it matters: No-shows are the #1 clinic complaint. A WhatsApp message 24 hours before an appointment ("Your appointment with Dr. X is tomorrow at 3pm — reply 1 to confirm, 2 to cancel") reduces no-shows by 30–50%. Every Egyptian patient has WhatsApp.

How to build:

- Sign up for [Twilio](#) → enable WhatsApp API (~\$0.005/message).
- On appointment creation, call Twilio API to send an immediate confirmation.
- Schedule a reminder 24 hours before using a Supabase Edge Function cron job.
- Handle replies: Twilio webhook → update appointment status in Supabase.

```
// whatsapp.service.ts
async sendAppointmentReminder(phone: string, doctorName: string, scheduledAt:
Date) {
  const message = `تذكير: موعدك مع ${doctorName} غداً ${format(scheduledAt, 'h:mm
a')}\nرد 1 للتأكيد أو 2 للإلغاء`;
  await fetch('https://api.twilio.com/2010-04-01/Accounts/.../Messages.json', {
    method: 'POST',
    headers: { Authorization: 'Basic ' + btoa(`${accountSid}:${authToken}`) },
    body: new URLSearchParams({
      From: 'whatsapp:+14155238886',
      To: `whatsapp:+2${phone}`,
      Body: message
    })
  });
}
```

Arabic UI with RTL support

Why it matters: Receptionists at small clinics often cannot use English-only software. An Arabic toggle is the difference between "my staff can't use this" and a sale.

How to build:

1. Install ngx-translate: `npm install @ngx-translate/core @ngx-translate/http-loader`
2. Create `src/assets/i18n/ar.json` with all UI strings in Arabic.

3. Toggle direction on the <html> element:

```
switchLanguage(lang: 'en' | 'ar') {  
  this.translate.use(lang);  
  document.documentElement.setAttribute('dir', lang === 'ar' ? 'rtl' : 'ltr');  
  document.documentElement.setAttribute('lang', lang);  
}
```

1. Add a language toggle button in the nav bar.
-

Printable Arabic prescription

Why it matters: Doctors write prescriptions by hand — slow and illegible. A one-click print-ready Arabic prescription is something they feel immediately in their daily workflow.

How to build: Add a print button to the prescription view. Use a print-specific CSS that hides the app chrome and renders a clean A5 layout:

```
printPrescription(prescriptionId: string) {  
  window.open(`/print/prescription/${prescriptionId}`, '_blank');  
  // Or use jsPDF – already in HealthSync  
}
```

The print template (Arabic, RTL):

- Clinic logo + name at top
 - Doctor name + specialization
 - Patient name + date
 - Medication table (name | dosage | frequency | duration)
 - Doctor signature line
-

Cash payment tracking

Why it matters: Small Egyptian clinics are almost entirely cash-based. They track revenue in a notebook. Replacing that notebook with a searchable, reportable digital log is an immediate daily improvement.

How to build: Add a "Mark as paid" button on each appointment card. This creates a row in the payments table and toggles the appointment's `paid` flag. Show a daily revenue total on the admin dashboard.

```
async markAsPaid(appointmentId: string, amount: number) {  
  await this.supabase.client.from('payments').insert({  
    appointment_id: appointmentId,  
    clinic_id: this.clinicId,  
    amount,  
    method: 'cash',  
    received_at: new Date().toISOString()  
  });  
  await this.supabase.client  
    .from('appointments')  
    .update({ paid: true, paid_at: new Date().toISOString() })  
}
```

```
.eq('id', appointmentId);  
}
```

Patient self-booking page

Why it matters: Clinics get most bookings by phone. A public URL (`healthsync.app/book/dr-ahmed`) lets patients book themselves. The clinic shares it in their Facebook bio and on a printed A4 at the front desk.

How to build:

- Create an Angular route `/book/:clinicSlug` with no auth guard.
- Fetch the clinic's doctors and their available slots.
- Patient fills: name, phone, selects doctor + time → creates appointment with `status: 'pending'`.
- Admin reviews and confirms pending bookings from the dashboard.

Tier 2 — Differentiators (build after first client)

Feature	What it does	How long
Waiting room queue display	Tablet/TV screen at <code>/queue/:clinicId</code> shows current patient number. Auto-refreshes via Supabase Realtime.	1 day
Daily schedule printout	Print/PDF of today's patient list for the doctor. Reuses jsPDF.	2 hours
No-show tracking	Auto-flag missed appointments. Increment <code>no_show_count</code> on patient. Show warning badge on profile.	1 day
Follow-up reminders	When a medical record sets a follow-up date, auto-schedule a WhatsApp reminder 2 days before.	2 days

Tier 3 — Skip for now

Do not build any of the following until you have at least 5 paying clinics. Building these before validation is wasted time.

- Video consultations / telemedicine
- Lab results integration

- Insurance claim management
- Patient mobile app
- Multi-branch clinic support
- Fawry / online payment collection

7. Full Tech Stack Reference

Layer	Tool	Why
Frontend	Angular 17 (already built)	You already know it, product is built
Database	Supabase (PostgreSQL)	Free tier, built-in auth, RLS for multi-tenancy
Authentication	Supabase Auth	Email/password, magic links, invite flow
File storage	Supabase Storage	For prescription PDFs, lab results later
Realtime	Supabase Realtime	Queue display, live appointment updates
WhatsApp	Twilio WhatsApp API	Industry standard, easy sandbox for testing
Payments	Stripe	Subscriptions, webhooks, hosted checkout
Deployment	Vercel (frontend already there)	Free tier, instant deploys from GitHub
Backend functions	Supabase Edge Functions	Webhook handlers, scheduled jobs (Deno runtime)
Email	Resend.com	Transactional emails (optional, free 3k/month)

Total monthly infrastructure cost at 0 paying clients: \$0 (All tools above have free tiers sufficient for the first 10–20 clinics)

8. Pricing Strategy

Egyptian market (EGP)

Plan	Price	Included
Trial	Free (14 days)	All features, 1 doctor
Starter	200 EGP/month	Up to 3 doctors, basic features

Plan	Price	Included
Pro	450 EGP/month	Unlimited doctors, WhatsApp reminders, analytics

UAE / Remote (USD)

Plan	Price	Included
Trial	Free (14 days)	All features
Starter	\$15/month	Up to 3 doctors
Pro	\$35/month	Unlimited + WhatsApp + priority support

Notes on pricing

- Do not offer a free tier permanently. Free users never convert and drain support time.
- Offer the first 3 beta clinics 3 months free in exchange for a testimonial and feedback.
- After 10 paying clinics, raise prices. Early clients are locked at their original rate (grandfather them).
- Never discount. Offer extended free trials instead of lower prices — discounting devalues the product permanently.

Realistic Timeline

Week	Milestone
Now	Show Vercel demo to 5 clinic owners. Collect objections.
Week 1–2	Supabase migration (database + services swap)
Week 3	Auth + clinic registration + multi-tenancy
Week 4	Stripe billing + trial gate + landing page
Week 4–5	WhatsApp reminders + cash payment tracking
Week 5–6	Arabic UI + printable prescription
Week 6–7	Self-booking page live
Month 2	First beta clinic onboarded (free)
Month 3	First paying client
Month 4–6	3–5 paying clinics, \$200–500 MRR

Stripe + Twilio + Vercel Target market: Small private clinics in Egypt (Cairo, Giza) and UAE